Manuscript 11: Lean 4 and Arb certification record

Best approximation of the Eulerian polar curve by convex Pascal limaçons and sharp Fourier truncation errors

Roberto Alfano, 7 November 2026

1. Scope.

This record states, for manuscript 11, which results are kernel-checked in Lean 4 and how every displayed decimal is certified in interval and Arb ball arithmetic [3]. It replaces the certification remarks formerly made in the text of the article (Introduction, Section 3 and Conclusions), the corresponding sentences of its Data Availability statement and of its declaration on generative AI, and the reference-list entry of the repository https://github.com/ralfanop/Certifications_Eulerian_polar-curve_computations. Equation and statement numbers are those of manuscript 11, which are those of V001. The detailed map from every claim to its script, log and Lean module is the README.md of the repository (its Sections 2–5 use the numbering of V478; its Section 5a gives the status in the numbering of V001).

2. Lean 4 formalisation.

Every numbered statement of the article, from Definition 2.1 to Remark 3.16, is formalised in Lean 4, with the exception of the numerical values and finite exact computations, which are certified by interval and exact integer arithmetic (Section 3 below), and of the meaning of $E(n,k)$ as the number of permutations with k ascents, which is classical [1, pp. 95–9] (see also [2, pp. 209–10]); in the formalisation $E(n,k)$ is defined by its recurrence. In particular:

- Lemma 3.2 is formalised for the polar curves as subsets of $\mathbb{C}$, including the Hausdorff-distance identity;
- Theorems 3.12 and 3.13 are formalised, including the convexity of Ω_h, Eq. (73), the uniqueness of the minimiser and the scalar-balance argument that turns $Q(L) > L$, $Q(U) < U$ into $L < h_* < U$;
- in Remark 3.16 the integer inequalities that give $\alpha_{669} > \vartheta_E > \alpha_{671}$ after Eq. (93) are also verified by kernel evaluation.

The library `EulerianCert`, in the directory lean/EulerianCert, is compiled with the Lean 4 toolchain `v4.35.0-rc3` and Mathlib commit `0272bbe1c093`; it is built with `lake exe cache get` and `lake build`, and `lake env lean AxiomCheck.lean` repeats the axiom audit. Two intermediate estimates of the proof of Lemma 3.15, Eqs. (88)–(90), are also proved verbatim in the library `V001` of lean/EulerianGrothendieck (Lean 4.24.0, Mathlib `f897ebcf`).

3. Certified decimals.

A displayed decimal is certified when a rigorous enclosure $[L, U]$ of the exact quantity lies inside the rounding cell of that decimal. Decimal values of exact quantities were certified by outward-rounded interval arithmetic (`mpmath.iv`) with explicit series remainders, and independently replayed in an

exact-integer fixed-point interval arithmetic written for this purpose (numerics/ia$_f$*ixed.py*); every displayed decimal is also certified in the ball arithmetic of Arb [3] (numerics/arb). In particular:

- the sharp uniform radial error (29) of Corollary 3.5 is certified to 80 decimal places;
- D_κ in (50) is enclosed by validated Taylor integration (order 30, 56 steps on $[0,\pi]$) of the arclength-matching equation $\varphi' = (L_L/L_E)\,\sigma_E(\theta)/\sigma_L(\varphi)$, where $\sigma = \sqrt{r^2 + r'^2}$, with Lagrange remainders bounded through a Picard a priori enclosure, in both interval arithmetics (numerics/dkappa$_c$*ert.py*, numerics/arb/dkappa);
- the decimals of Theorem 3.12 are certified (numerics/epsrad$_c$*ert.py*);
- h_* in (70) is certified correctly rounded to 80 decimal places (numerics/hstar$_c$*ert.py*, numerics/hstar$_c$*ertificate.json*, numerics/arb/hstar80). The 80-place decimal used in the proof of Theorem 3.13 is

$$\begin{aligned} d = 0.&0492979483\,2571833570\,7036623003\,9670122802 \\ &4147449936\,0414188045\,3238853121\,8517828559, \end{aligned}$$

  with $L = d - \frac{1}{2}10^{-80}$ and $U = d + \frac{1}{2}10^{-80}$; the checks $Q(L) > L$ and $Q(U) < U$ described in that proof are run on both interval backends, and the negative controls of numerics/hstar$_n$*egative$_c$ontrols.py* shift d by $\pm 10^{-80}$ and make the corresponding check fail;
- the integer comparison (93) of Remark 3.16 is checked by exact integer arithmetic (numerics/eulerian$_b$*racket.py*).

References.

[1] E. Deza, *Eulerian Numbers*, Selected Chapters of Number Theory: Special Numbers, Vol. 5, World Scientific, New Jersey, 2026. ISBN 9789819818655. https://doi.org/10.1142/14464.

[2] P. Flajolet, R. Sedgewick, *Analytic Combinatorics*, 1st edition, Cambridge University Press, Cambridge, 2009. ISBN 9780511801655. https://doi.org/10.1017/CBO9780511801655.

[3] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Comput.* **66** (2017), no. 8, 1281–1292. https://doi.org/10.1109/TC.2017.2690633.